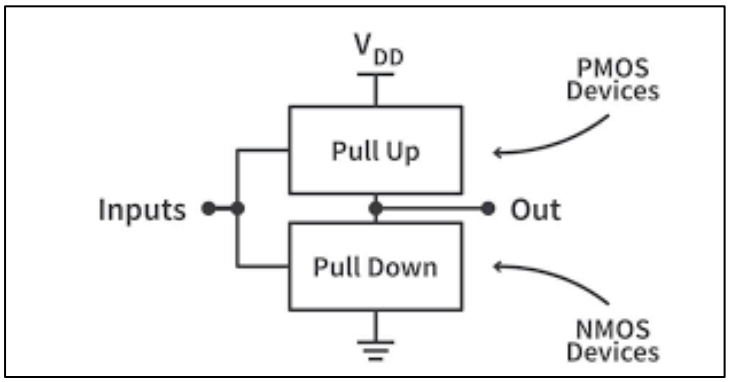
CMOS Fundamentals - Learn by Doing Transistor Circuits / SPICE Simulation Manual

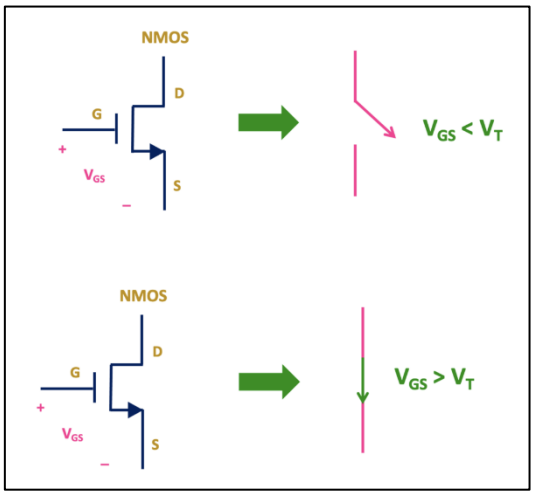
Document Title: Simulation Manual
Purpose: VLSI Foundations Teaching Aid
Version: v0.1

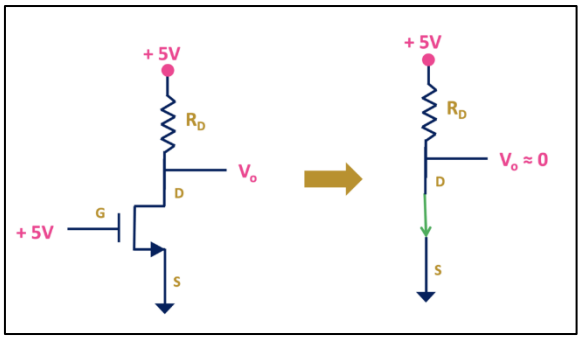
1 MOS Basics:

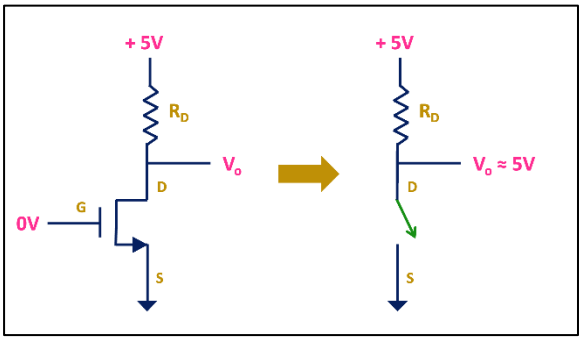
nMOS and pMOS are the two fundamental MOSFET device flavors that make up modern digital standard cells and custom logic. An nMOS turns ON for a high gate bias and is typically used to realize the pull-down network, while a pMOS turns ON for a low gate bias and is used in the complementary pull-up network. Paired together, they form complementary MOS (CMOS) logic, which is the basic circuit style behind mainstream microprocessor datapaths, control logic, memories, and interface circuitry.

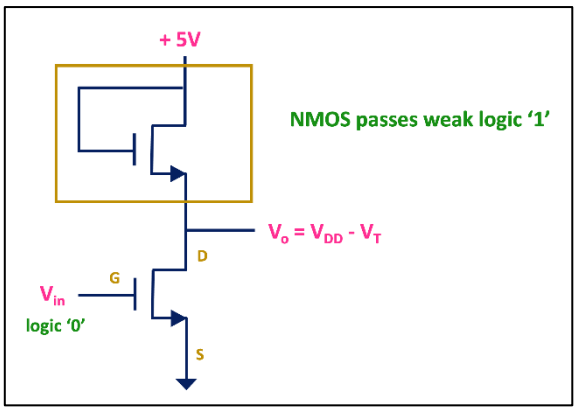


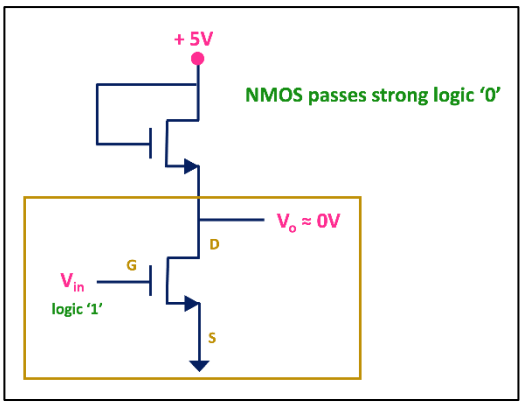
1.1 NMOS Pull-Down / Switch Behavior



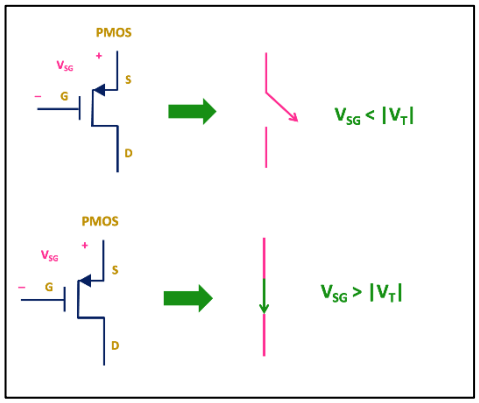


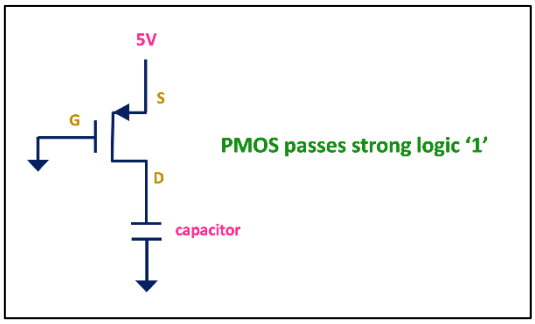


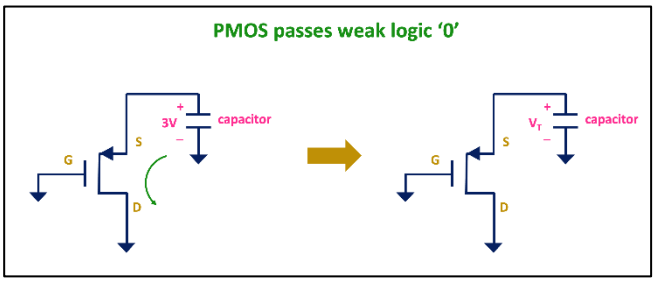




1.2 PMOS Pull-Up / Switch Behavior







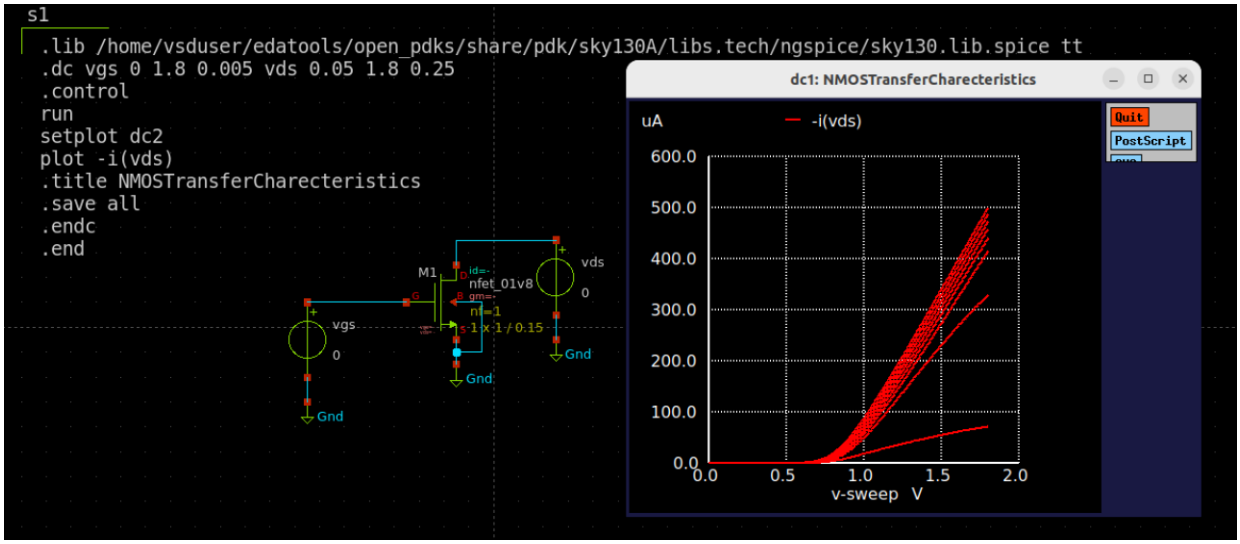
Generic Metrics to understand the efficiency of MOS devices

- Switching Threshold (VM): the input voltage at which output = input
- Noise Margins (NMH, NML): indicators of noise immunity
- Propagation Delays (tPHL, tPLH): delays during output transitions
- Rise and Fall Times (trise, tfall): speed of output transition edges
- Dynamic Power Consumption: estimated from average current drawn from VDD
- **dc sweep** VTC curve, VM, noise margins , VM, NMH, NML
- **tran** Time-based waveform of Vin/Vout tPHL, tPLH, trise, tfall, avg_power

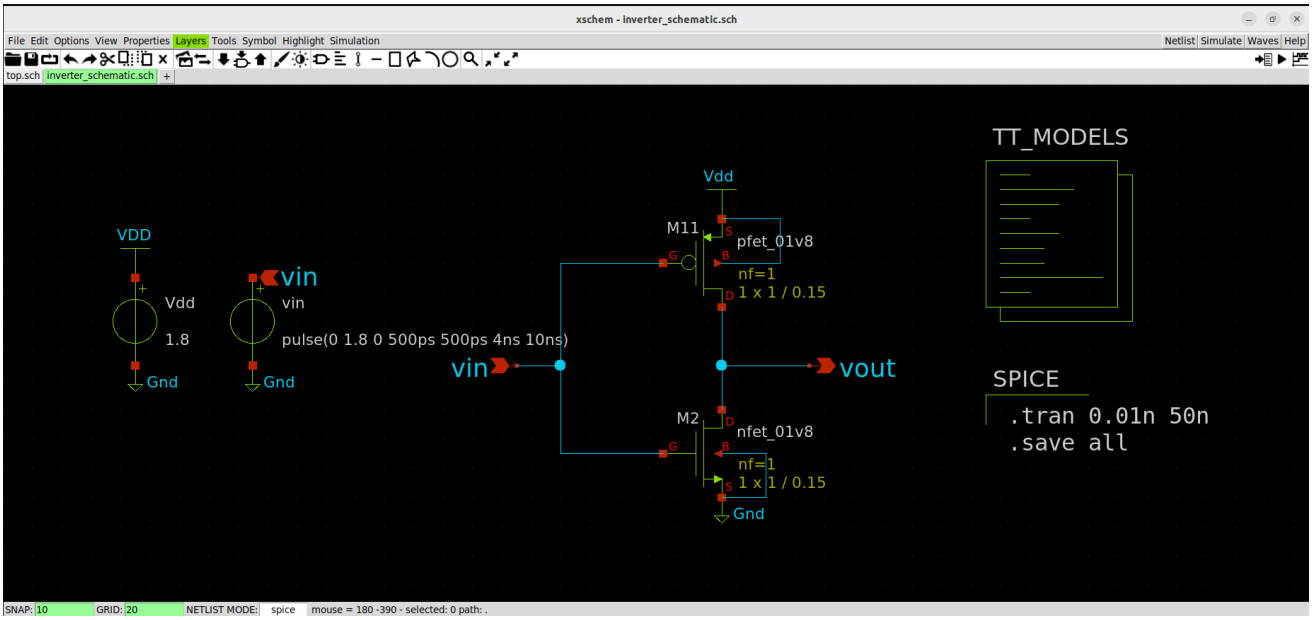
2 Workflow 1: Schematic Capture → SPICE Netlisting → Circuit Simulation

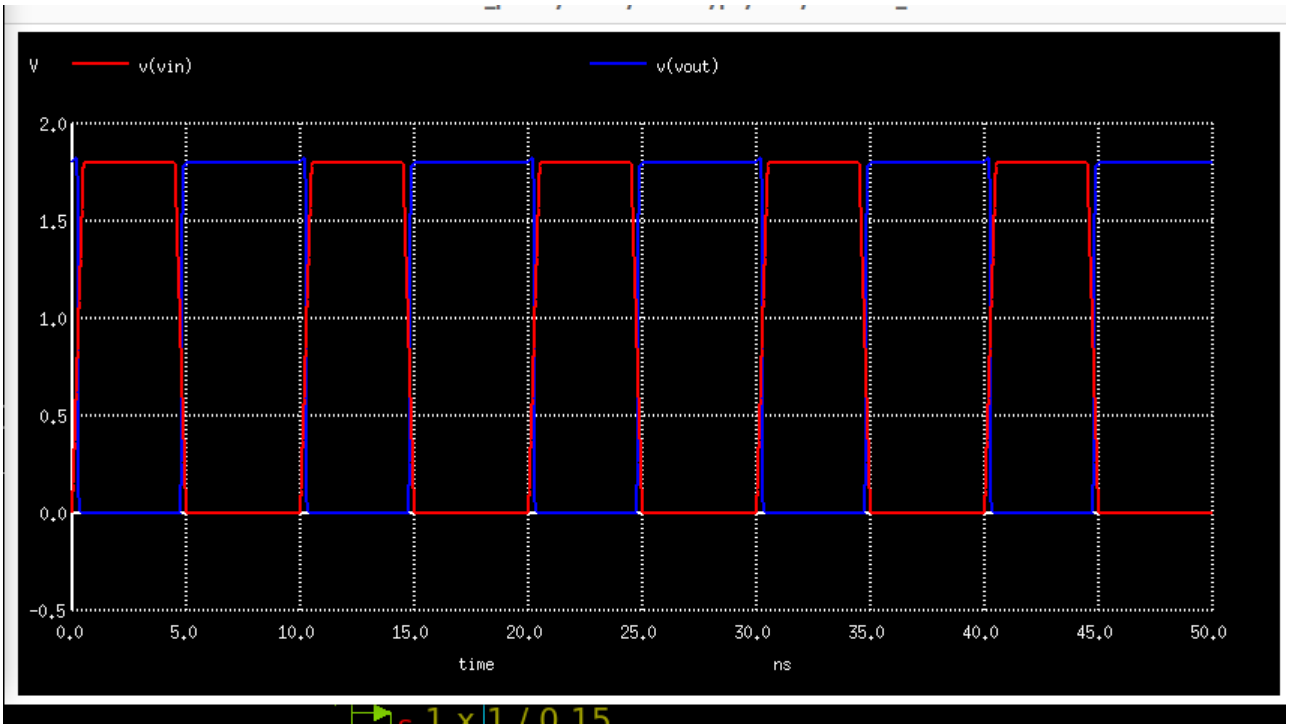
1. **Launch xschem and create a new schematic design**
   Open xschem, start a new schematic file, and save it with an appropriate circuit name for proper project organization.
2. **Instantiate the required symbols from the appropriate libraries**
   Place `nfet_01v8.sym` from the `xschem_sky130` library, and `vsource.sym` and `code_shown.sym` from the `xschem` devices library.
3. **Construct the circuit at schematic level**
   Interconnect the transistor, voltage sources, and reference ground nodes according to the intended circuit topology.
4. **Configure device-level and source-level parameters**
   Assign the required transistor dimensions such as width and length, and define the voltage source specifications including DC bias or time-varying input stimulus.
5. **Insert the simulation control block using `code_shown.sym`**
   Add the SPICE directives required for simulation, including model file inclusion, analysis commands, and any waveform or measurement instructions.
6. **Generate the SPICE netlist from the schematic database**
   Use xschem's netlisting function to translate the graphical schematic into a valid SPICE netlist for circuit simulation.
7. **Execute the generated netlist using ngspice**
   Run the circuit simulation through ngspice to evaluate the electrical behavior of the designed circuit under the defined test conditions.
8. **Visualize and interpret the simulation results using GAW**
   Open the generated waveform data in GAW and analyze key responses such as node voltages, switching behavior, and transient characteristics.

2.1 EXP:NMOS



2.2 EXP : CMOS


xschem - inverter_schematic.sch
File Edit Options View Properties Layers Tools Symbol Highlight Simulation
Netlist Simulate Waves Help
top.sch inverter_schematic.sch
TT_MODELS
Vdd
M11
pfet_01v8
nf=1
1 x 1 / 0.15
VDD
Vdd
1.8
Gnd
vin
vin
pulse(0 1.8 0 500ps 500ps 4ns 10ns)
Gnd
vin
vout
M2
nfet_01v8
nf=1
1 x 1 / 0.15
Gnd
SPICE
.tran 0.01n 50n
.save all
SNAP: 10
GRID: 20
NETLIST MODE:
spice
mouse = 180 -390 - selected: 0 path: .


V
v(vin)
v(vout)
2.0
1.5
1.0
0.5
0.0
-0.5
0.0
5.0
10.0
15.0
20.0
25.0
30.0
35.0
40.0
45.0
50.0
time
ns

3 Workflow 2: Direct SPICE Netlist Entry → Circuit Simulation

Spice simualation as required to represent a given circuit in a standard format for its analysis and execution. As mentioned earlier through spice simulation input waveform can be fed into the circuit for the analysis of the output. A number of spice tools are available for creating a spice deck, in our case ngspice is used. There are four basic section required for creating a spice deck of a circuit

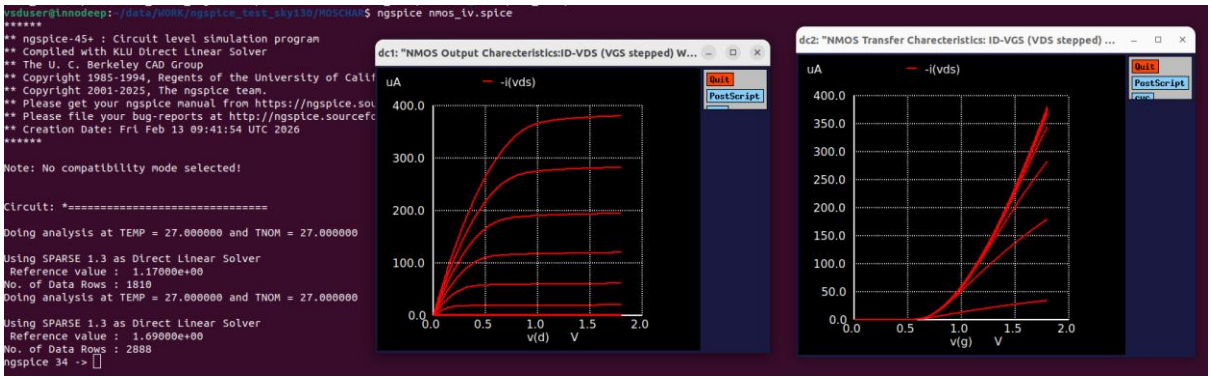
- Netlist decription: Obtain from the given circuit - Define input and output node, name nodes component connectivity, component values
- Including technology file: The technology file of any process node is provided by the foundry, contains information of all the parameters required in execution of spice deck
- Simulation commands: Required Simulation/operation to be executed.
- Control commands: The display or execution of the required output

1. **Write the circuit directly in SPICE syntax**
   Define components, node connections, sources, and device parameters manually in a text file.
2. **Include the required model files**
   Link the proper transistor or device models so the simulator can interpret the circuit correctly.
3. **Add simulation commands**
   Specify analyses such as `.op`, `.dc`, `.tran`, or `.ac` depending on what behavior you want to study.
4. **Run the netlist in ngspice**
   Execute the SPICE file directly from the simulator without drawing the schematic first.
5. **Observe and interpret results**
   Check node voltages, currents, waveforms, delays, and other performance metrics from the simulation output.

3.1 EXP:NMOS

```
*================================
* SKY130 NMOS IV Characteristics
* 1) ID vs VDS with VGS sweep
* 2) ID vs VGS with VDS sweep
*================================
.temp 27
.options savecurrents
* ---- PARAMS (edit only these) ----
.param W=5
.param L=2
* ---- Models ----
.lib
"/home/<username>/edatools/open_pdks/share/pdk/sky130A/libs.tech/ngspice/
sky130.lib.spice" tt
* ---- Bias sources ----
VDS d 0 0
VGS g 0 0
* ---- Device: D G S B ----
XM1 d g 0 0 sky130_fd_pr__nfet_01v8 w={W} l={L}
*-------------------------------------
* (1) Output characteristics: ID–VDS
* Sweep VDS, step VGS
*-------------------------------------
.dc VDS 0 1.8 0.01 VGS 0 1.8 0.2
.control
run
setplot dc1
* ID = -I(VDS) (sign fix)
plot -i(VDS) vs v(d) title "NMOS Output Charecteristics:ID-VDS (VGS stepped)
W={W} L={L}"
*-------------------------------------
```

```
* (2) Transfer characteristics: ID–VGS

* Create a NEW DC sweep (separate plot)

* Sweep VGS, step VDS

*-------------------------------------

.dc VGS 0 1.8 0.005 VDS 0.05 1.8 0.25

setplot dc2

plot -i(VDS) vs v(g) title "NMOS Transfer Charecteristics: ID-VGS (VDS stepped)
W={W} L={L}"

.endc

.end
```



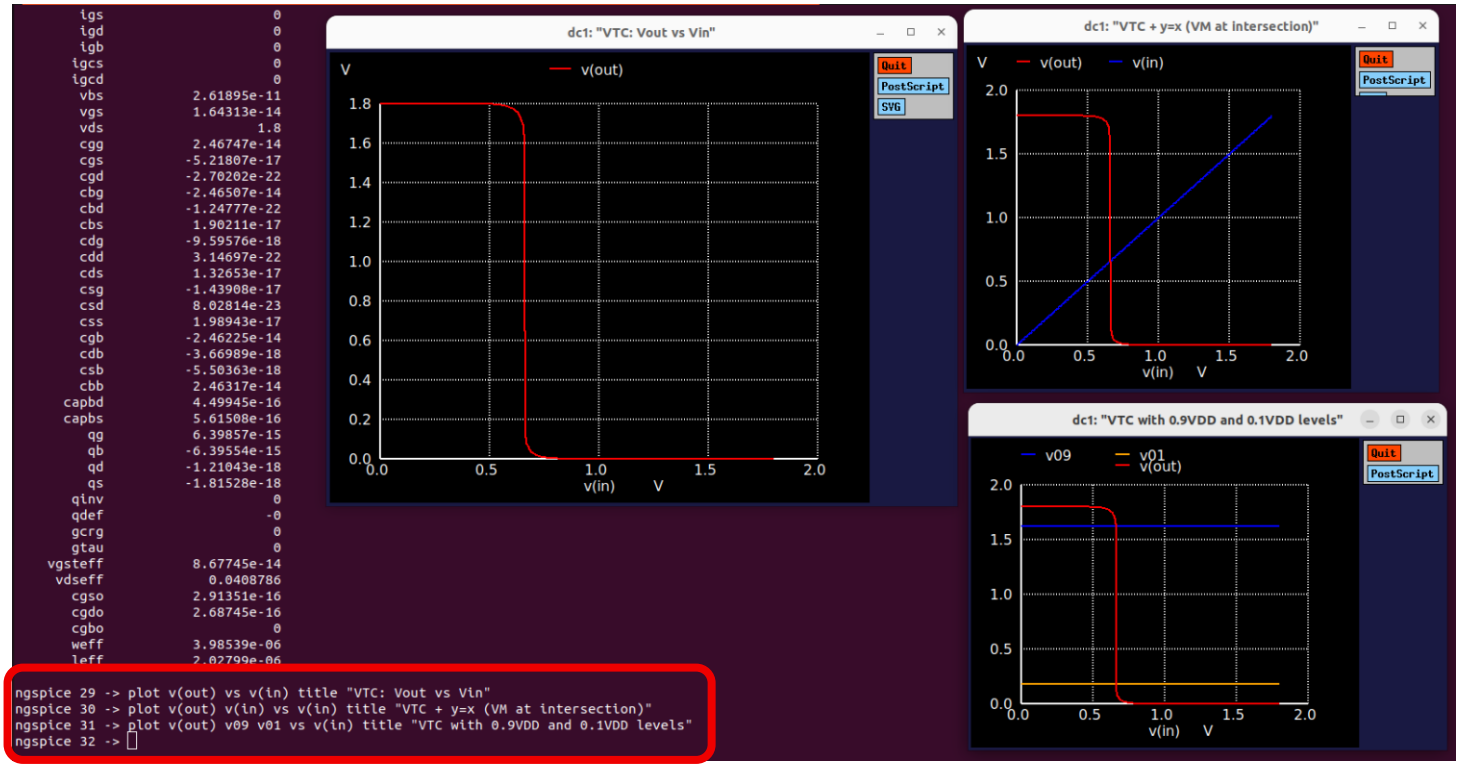
S

3.2 EXP: CMOS Inverter

```
*================================
* Unit CMOS Inverter : unitinv.spice
*================================
.temp 27
.options savecurrents
.param VDD=1.8
.lib
"/home/<username>/edatools/open_pdks/share/pdk/sky130A/libs.tech/ngspice/
sky130.lib.spice" tt
VDD vdd 0 1.8
VIN in  0 0
* name d g b s dev name w l
XM1 out in 0   0   sky130_fd_pr__nfet_01v8 w=2 l=2
XM2 out in vdd vdd sky130_fd_pr__pfet_01v8 w=4 l=2
.dc VIN 0 1.8 0.005
.control
run
setplot dc1
*================================ VTC================================
plot v(out) vs v(in) title "VTC: Vout vs Vin"
*================================ Add y=x line to show VM visually
plot v(out) v(in) vs v(in) title "VTC + y=x (VM at intersection)"
*====== Add 0.9VDD and 0.1VDD lines to show switching region=========
let v09 = 0.9*1.8
let v01 = 0.1*1.8
plot v(out) v09 v01 vs v(in) title "VTC with 0.9VDD and 0.1VDD levels"
.endc
.end
```

```
vsduser@innodeep:~/data/WORK/ngspice_test_sky130/MOSCHAR/FO1vsFo4$ ngspice unitinv.spice
******
** ngspice-45+ : Circuit level simulation program
** Compiled with KLU Direct Linear Solver
** The U. C. Berkeley CAD Group
** Copyright 1985-1994, Regents of the University of California.
** Copyright 2001-2025, The ngspice team.
** Please get your ngspice manual from https://ngspice.sourceforge.io/docs.html
** Please file your bug-reports at http://ngspice.sourceforge.net/bugrep.html
** Creation Date: Fri Feb 13 09:41:54 UTC 2026
******

Note: No compatibility mode selected!


Circuit: *==============================

Doing analysis at TEMP = 27.000000 and TNOM = 27.000000
Trash
Using SPARSE 1.3 as Direct Linear Solver
 Reference value :  1.40500e+00
No. of Data Rows : 361
ngspice 14 -> display all
    @m.xm1.msky130_fd_pr__nfet_01v8[ib]: current, real, 0 long
    @m.xm1.msky130_fd_pr__nfet_01v8[id]: current, real, 361 long
    @m.xm1.msky130_fd_pr__nfet_01v8[ig]: current, real, 0 long
    @m.xm1.msky130_fd_pr__nfet_01v8[is]: current, real, 0 long
    @m.xm2.msky130_fd_pr__pfet_01v8[ib]: current, real, 0 long
    @m.xm2.msky130_fd_pr__pfet_01v8[id]: current, real, 361 long
    @m.xm2.msky130_fd_pr__pfet_01v8[ig]: current, real, 0 long
    @m.xm2.msky130_fd_pr__pfet_01v8[is]: current, real, 0 long
    in                  : voltage, real, 361 long
    m.xm1.msky130_fd_pr__nfet_01v8#body: voltage, real, 361 long
    m.xm1.msky130_fd_pr__nfet_01v8#dbody: voltage, real, 361 long
    m.xm1.msky130_fd_pr__nfet_01v8#sbody: voltage, real, 361 long
    m.xm2.msky130_fd_pr__pfet_01v8#body: voltage, real, 361 long
    m.xm2.msky130_fd_pr__pfet_01v8#dbody: voltage, real, 361 long
    m.xm2.msky130_fd_pr__pfet_01v8#sbody: voltage, real, 361 long
    out                 : voltage, real, 361 long
    v-sweep             : voltage, real, 361 long [default scale]
    v01                 : notype, real, 1 long
    v09                 : notype, real, 1 long
    vdd                 : voltage, real, 361 long
    vdd#branch          : current, real, 361 long
    vin#branch          : current, real, 361 long
ngspice 27 -> show m.xm2.msky130_fd_pr__pfet_01v8[id]
No matching instances or models
ngspice 28 -> show m.xm2.msky130_fd_pr__pfet_01v8
 BSIM4v5: Berkeley Short Channel IGFET Model-4
     device m.xm2.msky130_fd_pr__
     model xm2:sky130_fd_pr__pfe
Trash
          l                 2e-06
          w                 4e-06
```

```
       gmbs            3.37146e-18
         gm            9.65522e-18
        gds            5.97053e-21
      vdsat               0.041111
        vth                1.03037
```


igs 0
igd 0
igb 0
igcs 0
igcd 0
vbs 2.61895e-11
vgs 1.64313e-14
vds 1.8
cgg 2.46747e-14
cgs -5.21807e-17
cgd -2.70202e-22
cbg -2.46507e-14
cbd -1.24777e-22
cbs 1.90211e-17
cdg -9.59576e-18
cdd 3.14697e-22
cds 1.32653e-17
csg -1.43908e-17
csd 8.02814e-23
css 1.98943e-17
cgb -2.46225e-14
cdb -3.66989e-18
csb -5.50363e-18
cbb 2.46317e-14
capbd 4.49945e-16
capbs 5.61508e-16
qg 6.39857e-15
qb -6.39554e-15
qd -1.21043e-18
qs -1.81528e-18
qinv 0
qdef -0
gcrg 0
gtau 0
vgsteff 8.67745e-14
vdseff 0.0408786
cgso 2.91351e-16
cgdo 2.68745e-16
cgbo 0
weff 3.98539e-06
ngspice 29 -> plot v(out) vs v(in) title "VTC: Vout vs Vin"
ngspice 30 -> plot v(out) v(in) vs v(in) title "VTC + y=x (VM at intersection)"
ngspice 31 -> plot v(out) v09 v01 vs v(in) title "VTC with 0.9VDD and 0.1VDD levels"
ngspice 32 ->
dc1: "VTC: Vout vs Vin"
V
v(out)
v(in) V
Quit
PostScript
SVG
dc1: "VTC + y=x (VM at intersection)"
V
v(out)
v(in)
v(in) V
Quit
PostScript
dc1: "VTC with 0.9VDD and 0.1VDD levels"
v09
v01
v(out)
v(in) V
Quit
PostScript